The Use Case Model

The Unified Process defines the ***Use Case Model*** within the Requirements discipline. The Use Case Model is the set of all use cases; it is a model of the system's functionality and environment.

An ***Actor*** is an object with a specific behavior (ex. a person, a computer system, etc).

A Scenario is a specific sequence of actions and interactions between actors and the system under discussion; it is also called a ***use case instance***.

The Use Case Model

[*Informal use case definition:*] A ***use case*** is a collection of related success and failure scenarios that describe actors using a system to support a goal.

[*RUP definition:*] A ***use case*** is a set of use case instances, where each instance is a sequence of actions a system performs that **yields an observable result of value** to a particular actor.

A key attitude in use case work is to focus on the question "How can using the system provide *an observable result of value* to the user, or fulfill their goals ?", rather than thinking of system requirements in terms of a "laundry list" of features or functions.

The Use Case Model

Why this ***"observable result of value ?".*** The software industry has a long list of failed projects that did not deliver what users really needed. The feature and function list approach to capturing requirements can contribute to this negative outcome because it does not encourage the stakeholders to consider the requirements in a larger context of using the system in a scenario to achieve some observable results of value, or some goal.

The Use Case Model

Use cases define a step-by-step sequence of actions between the user and the system. Organizations are rapidly adopting use cases as a means to communicate requirements because they:

1. Are easier to create, read, and understand than traditional functional specifications.

2. Show how the system will work from the users' perspective rather than the system's perspective.

3. Force us to think about the end-game: What is the user trying to accomplish by using the system?

The Use Case Model (continued)

4. Require us to define how the system should work, step-by-step.

5. Provide an excellent basis for building test cases and helping to ensure that these are built before the code is written.

6. Provide a common requirements "language" that's easy for stakeholders, users, analysts, architects, programmers, and testers to understand.

Use Cases vs. Functional Requirements

1. ***Use cases are requirements***. Primarily they are functional requirements that indicate what the system should do. Use cases are text documents, not diagrams, and use case modeling is primarily an act of writing text, not drawing diagrams. However, the UML defines a use case diagram to illustrate the name of the use cases and actors and their relationships.

2. ***Use cases are NOT ALL of the requirements.*** Use cases don't detail external interfaces, data formats, business rules and complex formulae. Typically, use cases represent no more than a third of all system requirements.

Use Case Name: **Process Sale**

Primary Actor: Cashier

Stakeholders and Interests:

1) Cashier: wants accurate fast entry and no payment errors as cash drawer shortages are deduced from his/her salary.

2) Salesperson: wants sales commissions updated.

3) Customer: wants purchase and fast service with minimal effort. Wants proof of purchase to support returns.

4) Company: wants to accurately record transactions and satisfy customer interests. Wants some fault tolerance to allow sales capture even if server components (ex. remote credit validation). Wants automatic and fast update of accounting and inventory.

Use Case Name: **Process Sale**

5) Tax Agencies: want to collect taxes from every sale.
6) Payment Authorization Service: wants to receive digital authorization requests in the correct format and protocol. Wants to accurately account for their payables to the store.

Preconditions: Cashier is identified and authenticated

Postcondition: Sale is saved. Tax is correctly calculated. Accounting and inventory are updated. Commissions recorded. Receipt (invoice) is generated. Payment authorization approvals are recorded.

Use Case Name: **Process Sale**

Main Success Scenario (Basic Flow):

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price and running total. *Cashier repeats steps 3-4 until indicates done (no more purchased items)*
5. System presents total with taxes calculated.
6. Cashier tells Customer the total and asks for payment.

Use Case Name: **Process Sale**

Main Success Scenario (Basic Flow – continued):

7. Customer pays and System handles payment.
8. System logs completed sale and sends sale and payment information to the external Accounting System and Inventory System.
9. Customer leaves with receipt and goods.

Use Case Name: **Process Sale**

Extensions (Alternate Flows):

*a At any time, System fails:

To support recovery and correct accounting, ensure all transaction sensitive state and events can be recovered from any step of the scenario.

1. Cashier restarts System, logs in and requests recovery of prior state.
2. System reconstructs prior state.

   2a. System detects anomalies preventing recovery:

      1. System signals error to the Cashier, records the error and enters a clean state.
      2. Cashier starts a new sale.

Use Case Name: **Process Sale**

Extensions (Alternate Flows - continued):

3a. Invalid identifier:

1. System signals error and rejects entry.

3b. There are multiple of same item category:

1. Cashier can enter item category identifier and the quantity.

3-6a. Customer asks Cashier to remove an item from the purchase:

1. Cashier enters item identifier for removal from sale.
2. System displays updated running total.

3-6b. Customer asks Cashier to cancel sale:

1. Cashier cancels sale on System.

Use Case Name: **Process Sale**

Extensions (Alternate Flows - continued):

3-6c. Cashier suspends the sale:

1. System records the sale so that it is available for retrieval on any POS terminal.

Use Case Name: **Process Sale**

Special Requirements:

Touch screen User Interface on a large flat panel monitor. Text must be visible from 1 meter.

Credit authorization response within 30 seconds in 90% of the time.

Robust recovery when access to remote services is failing

Language internationalization for the displayed text.

Technology and Data Variations List:

2a. Item identifier entered by bar code laser scanner or keyboard.

Semantics of the Use Case Components

1. Stakeholders

A Stakeholder is someone or something that has an interest in the behavior of the Use Case.

There are 2 types of stakeholders:

1. Stakeholders that interact directly with the system (ex. Primary Actor)
2. Stakeholders that **never** interact with the system directly. They are called ***OFFSTAGE, TERTIARY*** or ***SILENT ACTORS*** (ex. Tax Agencies)

Semantics of the Use Case Components

1. Stakeholders - continued

Any IT system operates a contract between stakeholders, with the use cases detailing the behavioral parts of that contract.

Conclusion: the content of any Use Case should be that which satisfies **ALL** the stakeholders' interests.

Semantics of the Use Case Components

2. Primary Actor

The **Primary Actor** is the stakeholder that calls on the system to deliver one of its services.

The Primary Actor is often, but not always, the actor who triggers the use case. Usually, the use case starts because the Primary Actor sends a message, pushes a button, enters a keystroke, or in some other way initiates a story.

Alternatively, if the use case is triggered by time, then it is not the Primary Actor who initiates the use case.

Semantics of the Use Case Components
2. Primary Actor - continued

Here are the advantages of keeping an extensive Primary Actors list:

1. It focuses the minds of the development team on those people who will use the system. As the requirements document contains the job description and the skills of the expected Primary Actors, the user interface and system designers use this information in order to match the system to that expertise.
2. It sets up the structure for the actor-goal list, which can be used to prioritize and partition the development work.
3. It will be used to partition a large set of use cases into packages that can be given to different design teams.

Semantics of the Use Case Components

3. Supporting Actors

A Supporting Actor is an external actor that provides a service to the system under design (for ex. a printer, a web service, etc).

Supporting actors identify the external interfaces of the system under development and the protocols that cross those interfaces.

Semantics of the Use Case Components
4. Preconditions and Postconditions

Preconditions state *what must always be true* before the beginning of a use case scenario. Preconditions are **NOT tested** within the use case; they are conditions assumed to be true. Generally, a precondition indicates that some other use case has already been executed and its result is the precondition (for ex. the user is already logged in and has been authenticated.

A **Postcondition** states what must be true on successful completion of the use case – either the main scenario or any of the alternate paths. Any postcondition (or success guarantee) should meet the needs of all stakeholders (ex. “all data are saved in a database”)

Semantics of the Use Case Components

5. Main Success Scenario

The Main Success Scenario, known also as the "**Happy Flow**", it describes the typical success path that satisfies the interests of the stakeholders.

Important note: The Main Success Scenario does not include any conditions or branching! Consequently, all conditional and branching statements are located in the **Extension** sections.

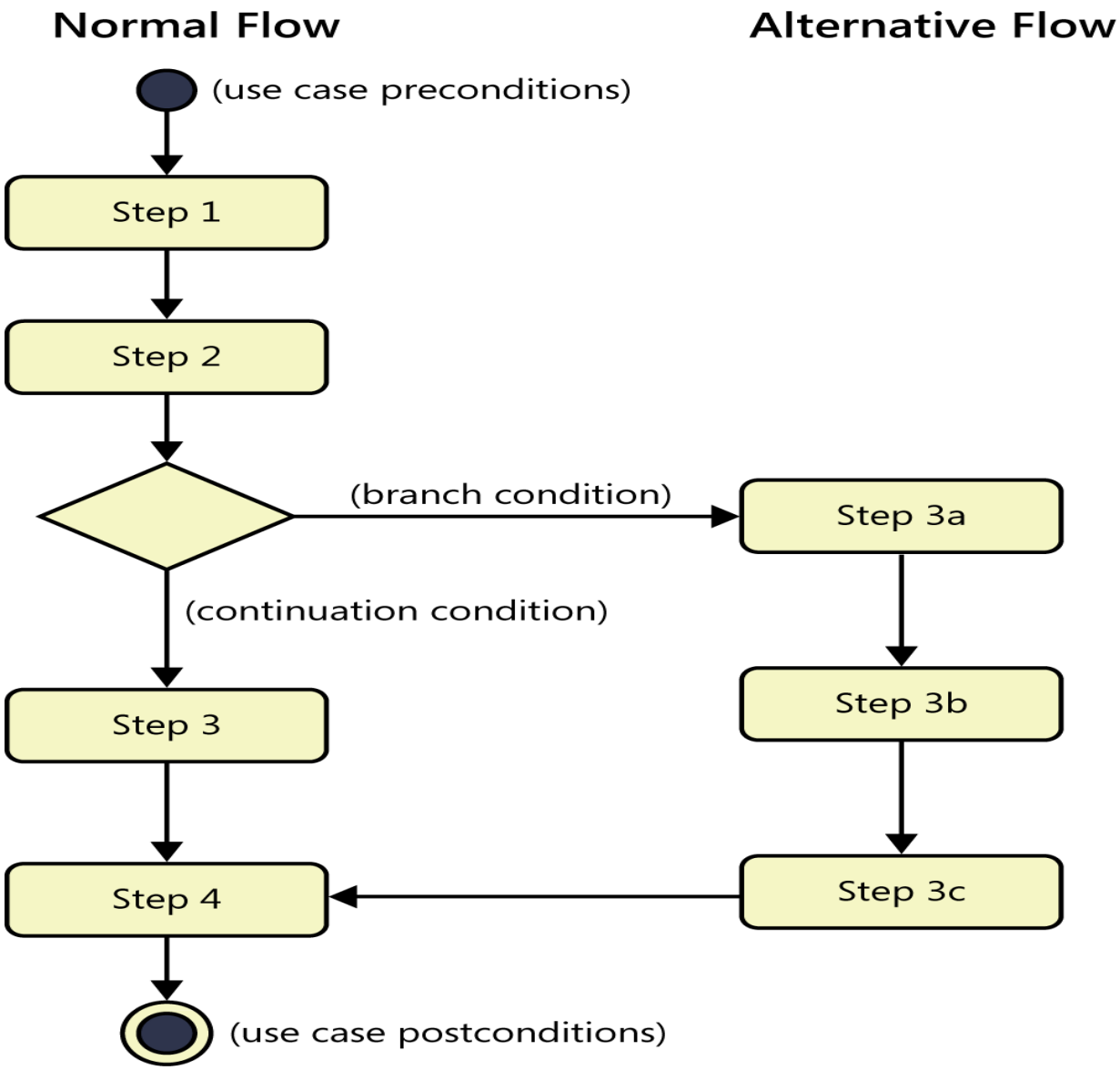
6. Extensions (Alternate Flows)

Extensions indicate all the other scenarios (except the main success scenario) or branches, both success and failure. Extension scenarios are branches from the main success scenario, and therefore are numbered and notated with respect to it:

Main Success Scenario	Extension
3.Cashier enters item identifier.	3a. Invalid identifier: 1. System signals error and rejects entry

From the above example, the Extension's number "**3a.**" shows that this branching occurs at step 3 in the Main Success Scenario, where the item identifier is validated.

6. Extensions (Alternate Flows) Handling



6. Extensions (Alternate Flows) - Guidelines

Any extension has 2 parts:

1. The condition (ex. *"Invalid identifier:"*)
2. The handling (ex. *"1.System signals error and rejects entry"*)

Guidelines for writing quality extensions:

Guideline 1: write the extension condition as something that can be **DETECTED** by the system or an actor, not just as something that happened.

Wrong	OK
1.Customer forgets PIN */* system cannot detect this !!!*	**1. PIN entry time-out** */* system can detect this !!!*

6. Extensions (Alternate Flows) - Guidelines

Guideline 2: always put a colon “(:)” after the condition to make sure the reader doesn’t think it is an action step. This simple notation can save many mistakes.

Wrong	OK
3a. Invalid identifier */*missing “:”*	**3a. Invalid identifier :**

6. Extensions (Alternate Flows) - Guidelines

Guideline 3: extension handling can be summarized in one step, or alternatively, as a sequence. The following example illustrates a notation used to indicate that a condition can arise within a range of steps.

Ex.

3-6a. Customer asks Cashier to remove an item from the purchase:

1. Cashier enters item identifier for removal from sale.
2. System displays updated running total.

The above example says that the condition *("Customer asks Cashier to remove an item from the purchase:")* can happen at any time during the steps 3.;4;5; or 6.; from the Main Success Scenario.

For both success and extensions scenarios there is a common structure consisting of the following items:

1. Conditions under which the scenario runs. For the success scenario, this is represented by precondition and the trigger. For the extension scenario, this is the extension condition.
2. A goal to achieve. For the success scenario this is the use case name. For the extension scenario, this is either to complete the use case goal or to rejoin the main success scenario after handling the condition (exception).
3. A set of action steps. This is the scenario body and it follows a set of common rules.
4. An end condition which is the goal in case of the success scenario, or an achieved or abandoned goal for the extension scenario.
5. A possible set of extensions written as scenario fragments.

7. Action Steps - Guidelines

Several guidelines, important for writing quality ("professional") use cases will be investigated.

Guideline 1: Use Simple Grammar

It is highly recommended that all use case steps to be written based on the following pattern:

"***Subject … verb … direct object … prepositional phrase***"

Ex:

System calculates VAT from the total balance.

7. Action Steps – Guidelines

Guideline 2: Show Clearly "Who Has the Ball"

In most sports, at one moment, only one player has the ball. Similarly, in all use case steps, only one actor is responsible for a certain action ("has the ball"). The actor plays the subject role from guideline 1.

Wrong	OK
2. Get ATM card and PIN No.	2. Customer inserts the ATM card and keys the PIN No.

7. Action Steps – Guidelines

Guideline 3: Show the Actor's Intent, NOT the Movement

Describing the user's presumed movements in operating the system's User Interface is one of the most common and severe mistakes in use case writing. The mistake is known as "*writing goals at a too low level*".

Wrong	OK
1.System asks for name 2.User enters name 3.System asks for address 4. User enters address 5.User clicks "Ok" 6.System displays user's profile	1.User enters name, address 2.System displays user's profile

7. Action Steps – Guidelines

Guideline 4: Use "Validate" instead of "Check whether …"

The reason behind this guideline is that "Checks whether" is often replaced with "IF" as a shortcut. "IF" means branching, which is in contradiction with the use case philosophy of "*NO branching within a main success scenario or extension*"

Wrong	OK
4. System checks whether the password is correct.	**4. System verifies that the password is correct.**

8. Goals and Scope of a Use Case

Two problems are investigated in the following:

1. How should use cases be discovered ?
2. Tasks can be grouped at many levels of granularity. At what level and scope should use cases be expressed ?

In order to answer question no. 1, we need to answer first the second question. Use cases can be defined at different levels depending on the system boundary, actors and goals.

Question: *"what is the useful level to express use cases for application requirements analysis ?"*.

Answer: use cases should be expressed at the level of ***Elementary Business Processes (EPB)***.

8. Goals and Scope of a Use Case

Definition: an **Elementary Business Processes (EPB)** is a task performed by one person, in one place, at one time, in response to a business event, which adds measurable business value and leaves the data in a consistent state.

Recommendation: a common mistake in use case writing is to define use cases at too low levels; that is, as a single step, subfunction or subtask within an EPB. For example, in a database context, defining use cases at **CRUD** (**C**reate, **R**ead, **U**pdate, **D**elete) level is in most cases wrong.

8. Goals and Scope of a Use Case

Based on these considerations, we can accurately answer the first question. Actors have goals (or needs) to fulfill. An EPB-level use case is called a ***user goal-level use case***, in order to emphasize that it serves (or should serve) to fulfill a goal of a system user, or the primary actor. Consequently, in order to identify (discover) use cases, we should follow the following recommended procedure:

1. *Find the user goals.*
2. *Define a use case for each goal.*

9. Finding Primary Actors, Goals and Use Cases

The procedure of identifying Primary Actors, Goals and Use Cases is summarized in the next 4-step algorithm:

1. *Choose the system boundary.*
2. *Identify the primary actors.*
3. *For each primary actor, identify their user goals. Raise these goals to the highest user goal level that satisfies the EBP guideline.*
4. *Define use cases that satisfy user goals; name them according to their goal.*

9. Use Case Diagrams

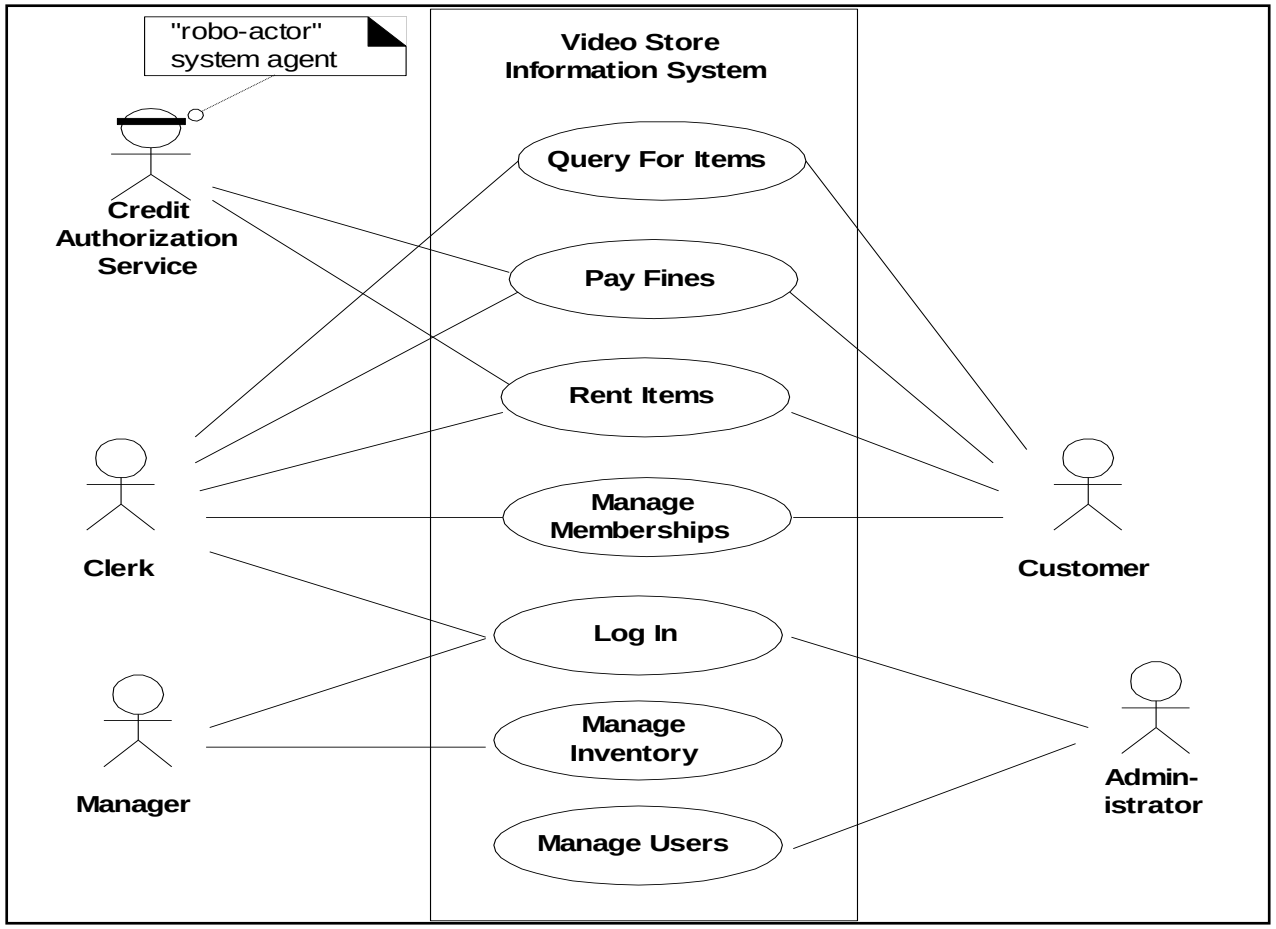
UML provides use case diagram notation to illustrate the names of the use cases and actors and he relationships between them.

It is important to remember that:

1. use case diagrams and use case relationships *are secondary in use case work*.
2. use cases are primarily text documents. *Doing use case work means to write text, not just draw UML diagrams !*
3. a common sign of nocive (or academic) use case modeling is a preoccupation with use case diagramming and use case relationships, rather than writing text. An acceptable trade-off should be to draw a simple use case diagram in conjunction with an actor-goal list, like in the figure above.

9. Use Case Diagrams

Example: a use case diagram associated to a Video Store Information system. It helps for a better understanding of the involved actors, their goals and their relationship. **This diagram should accompany the use case, not replace it !**

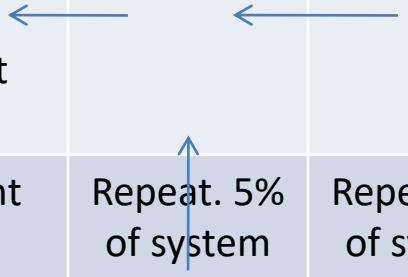


9. Use Cases and the Unified Process

Use cases are vital and central to the Unified Process, which encourages use-case driven development. This has the following implications:

1. Requirements are primarily recorded in use cases; the sum of all use cases generates the use case model.
2. Use cases are an important part of the iterative planning, based on choosing complete/incomplete use cases scenarios. Use cases are a key input for estimation.
3. Use cases realizations drive the design, a design based on collaborating objects and subsystems in order to perform or realize the use cases.
4. Use cases influence the organization and the content of user manuals.

Discipline	Artifact	Inception [1 week]	Elab. 1 [4 weeks]	Elab. 2 [4 weeks]	Elab. 3 [3 weeks]	Elab. 4 [3 weeks]
Requirements	Use Case Model	2 days Req workshop. Most UC identif. 10% UC written in detail.	30% UC written in detail.	50% UC written in detail.	70% UC written in detail.	70-80% UC written in detail. The rest done during construct.
Design	Design Model	none	Design a small set of high risk architect. significant req.	repeat	repeat	Repeat. High risk architect. significant req. are stable.
Implement.	Implemen. Model	none	Implement these	Repeat. 5% of system done.	Repeat. 5% of system done.	15% of system built.
Project Management	Software Developm. Plan	vague estimates	Estimations start to take shape.	Better estimates	Even better estimates	project duration, milestones, effort, cost estimates.



Use cases influence most UP Artifacts

